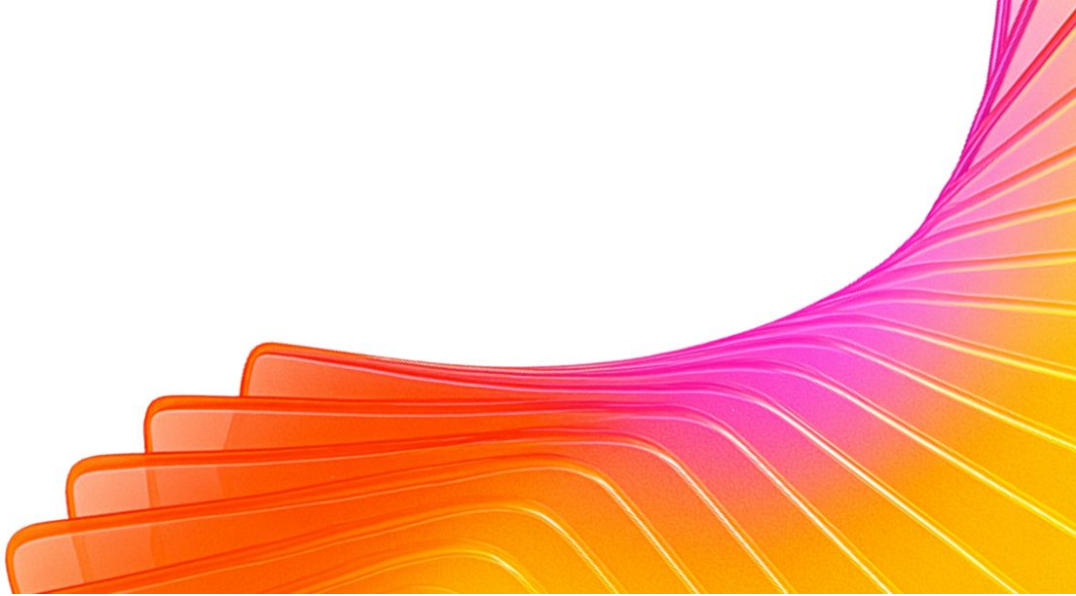
Google Cloud H2S

Gen AI Academy

APAC Edition

Build in APAC. Build for the world.



Participant Details

a. Participant Name: Kairux Labs - Duong Van Doan
b. Problem Statement: NVIDIA Track — Autonomous Multi-Agent Decision Intelligence for Smart Cities (CityOS / Civitas AI)

Brief about the idea

CityOS is an autonomous multi-agent decision-intelligence platform for smart cities. A goal-driven **Planner** breaks an incoming situation (e.g. a flash-flood warning) into a dependency-aware task graph, dispatches specialized **Traffic, Environment, Emergency, Citizen, and Knowledge agents** in parallel, reflects on their confidence, and produces an explainable, **human-approved** recommendation.

It is grounded by a real **knowledge graph** — OpenStreetMap, Wikipedia, Wikidata, GeoJSON boundaries, and government PDFs indexed into **Neo4j + Qdrant** — and reasons with **NVIDIA Nemotron** models served through OpenRouter, behind a single **AI Gateway** that can swap the backing provider without touching a single agent.

Operators watch it all happen in a live **Mission Control** dashboard — city map, agent graph, AI Copilot, and a decision report they approve or reject.

1. Approach & NVIDIA integration

We selected the NVIDIA problem statement. All reasoning — planning, embedding, reranking, and content-safety — runs on **NVIDIA Nemotron** models (Nemotron 3 Ultra, Llama-Nemotron-Embed-VL, Llama-Nemotron-Rerank-VL, Nemotron 3.5 Content-Safety) via OpenRouter, unified behind one AI Gateway so no agent talks to a model API directly. Google Gemini remains as an automatic fallback for resilience.

2. Real-world problem & impact

City operators juggle isolated dashboards that show data but cannot reason, coordinate, or recall institutional knowledge during fast-moving events (floods, hazardous AQI, mass events). CityOS turns raw sensor and knowledge data into a single explainable recommendation an operator can approve in seconds — cutting response time while keeping a human in the loop for every high-risk decision.

3. Core architecture & workflow

Goal → Planner (task DAG) → parallel agent workers → Reflection (confidence check) → RAG retrieval (Qdrant top-20 → Nemotron rerank → top-5) → Decision → human approval → workflow execution. Every step is broadcast live over WebSocket to Mission Control.

Opportunities

- How different is it from any of the other existing ideas?
- USP of the proposed solution

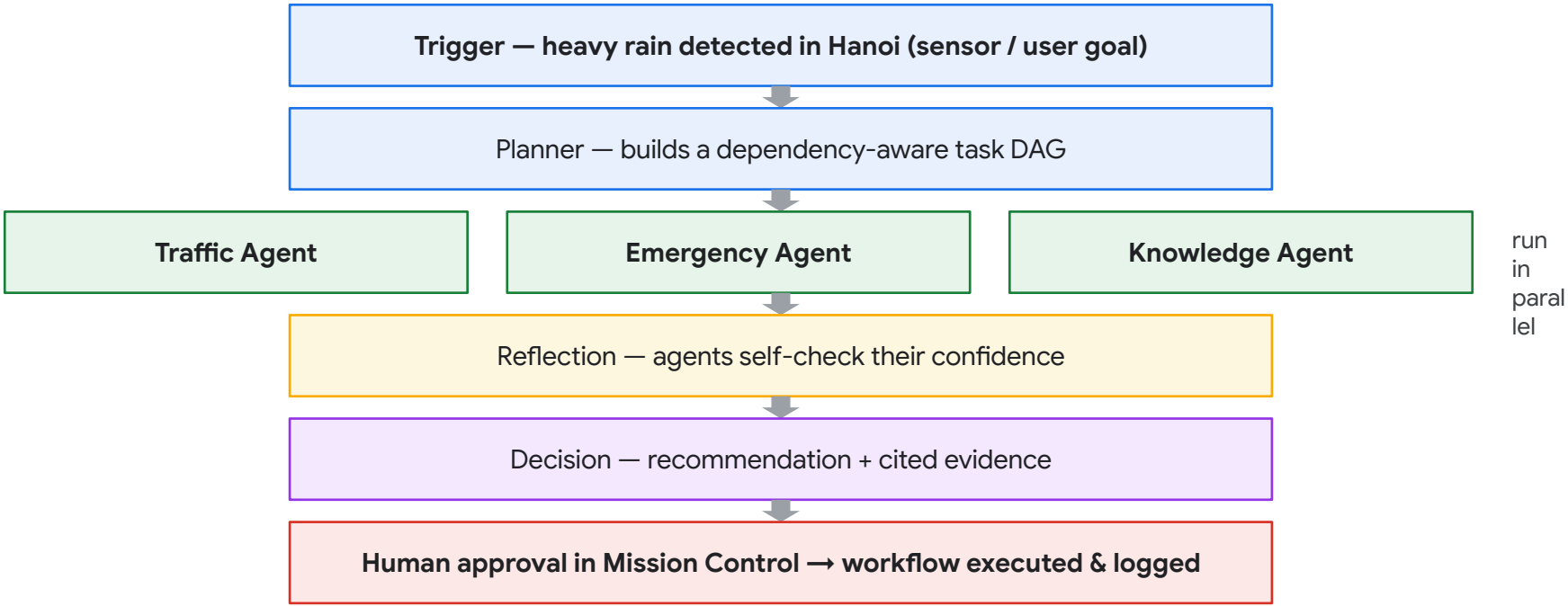
Most smart-city tools are read-only dashboards: they visualize data but cannot reason about it, coordinate across domains, or recall institutional knowledge.

- **True multi-agent reasoning, not another chart** — a Planner + Reflection loop that adapts its own task graph mid-run
- **A real knowledge graph (Neo4j + Qdrant), not keyword search** — 5 heterogeneous sources fused into one retrievable graph
- **Provider-agnostic AI Gateway** — swap NVIDIA → NIM → Vertex → Ollama by editing one file, zero agent changes
- **Guardrails by design** — content-safety pre/post-checks plus mandatory human approval on every high-risk decision

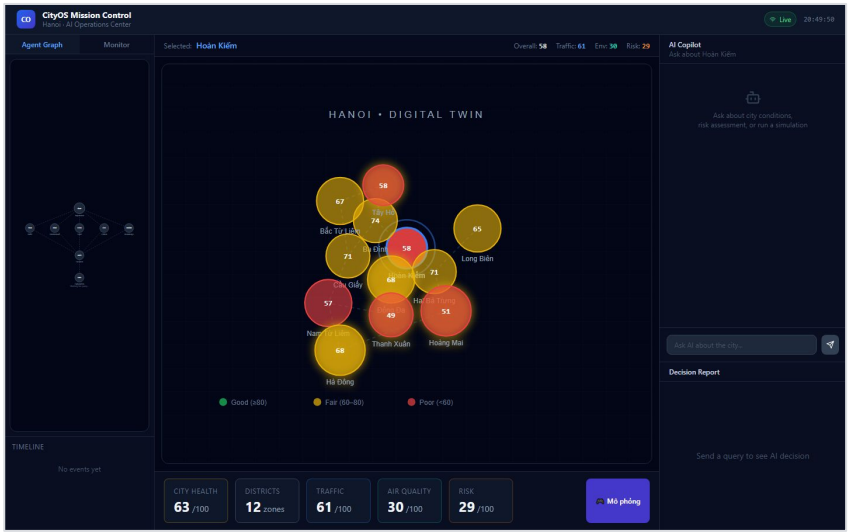
List of features offered by the solution

- **Goal-driven multi-agent runtime** — Planner → parallel Workers → Reflection → Decision → human approval
- **RAG knowledge layer** — OpenStreetMap, Wikipedia, Wikidata, GeoJSON, and government PDFs in Neo4j + Qdrant
- **NVIDIA Nemotron AI Gateway** — planning, embedding, reranking, content-safety, with automatic Gemini fallback
- **Live Mission Control dashboard** — city map, agent DAG, AI Copilot chat, decision timeline
- **What-If Simulator** — heavy rain / air pollution / major event / heatwave scenarios with live AI predictions
- **Real-time ingestion** — weather + AQI refreshed every 15 minutes across all 12 Hanoi districts
- **Human-in-the-loop safety gate** — every low-confidence or high-risk decision requires operator approval

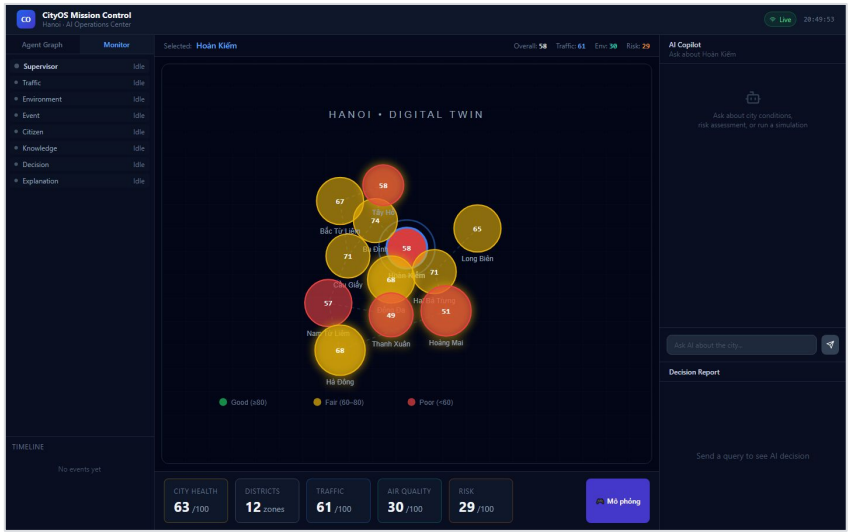
Process flow diagram or Use-case diagram



Wireframes/Mock diagrams of the proposed solution

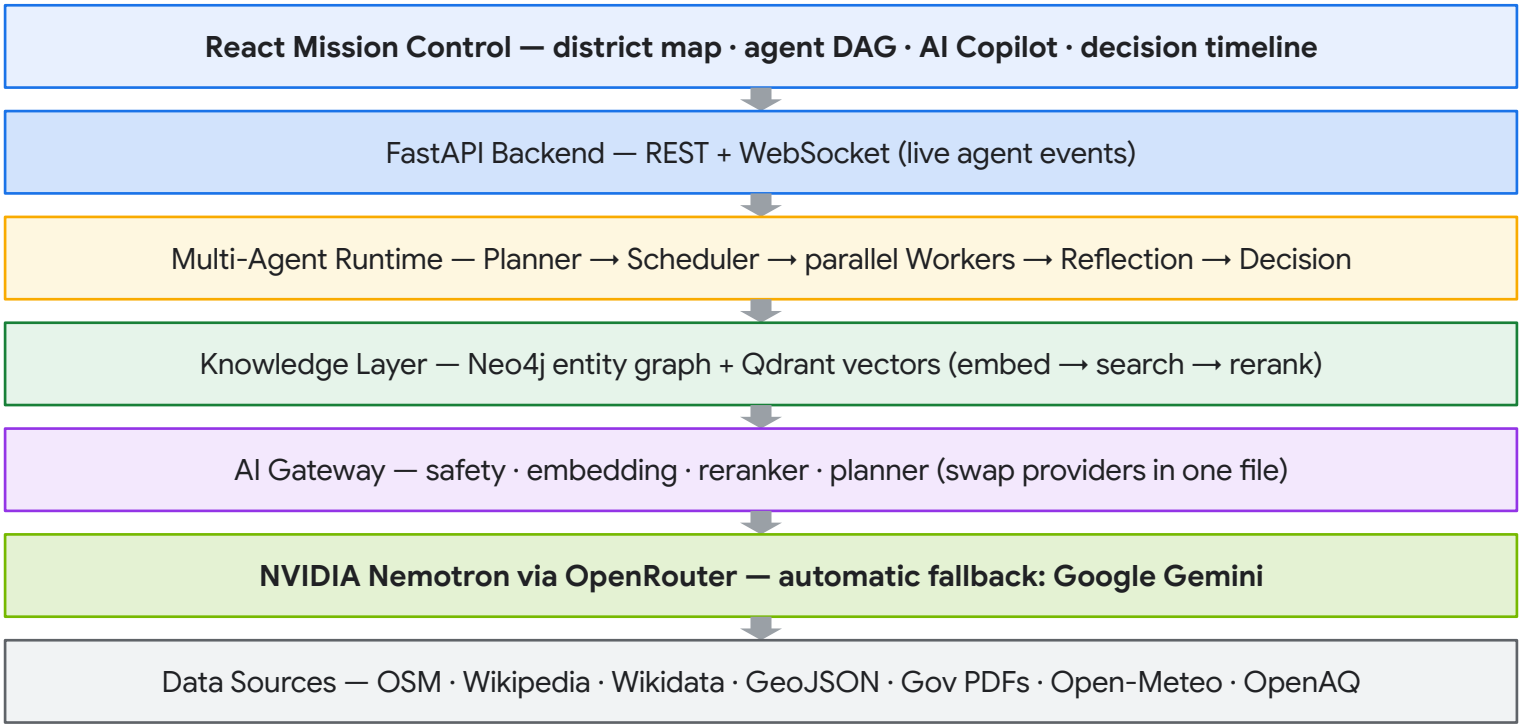


Digital Twin — live district health map



Agent Monitor — 8-agent pipeline status

Architecture diagram of the proposed solution:



Technologies / Google / Nvidia Services used in the solution

(Why did you choose your specific AI stack and system design, and how does it support scalability and real-world deployment?)

NVIDIA (via OpenRouter, free tier): Nemotron 3 Ultra (planning) · Llama-Nemotron-Embed-VL-1B-V2 (embedding) · Llama-Nemotron-Rerank-VL-1B-V2 (RAG reranking) · Nemotron 3.5 Content-Safety (guardrails, pre + post-check)

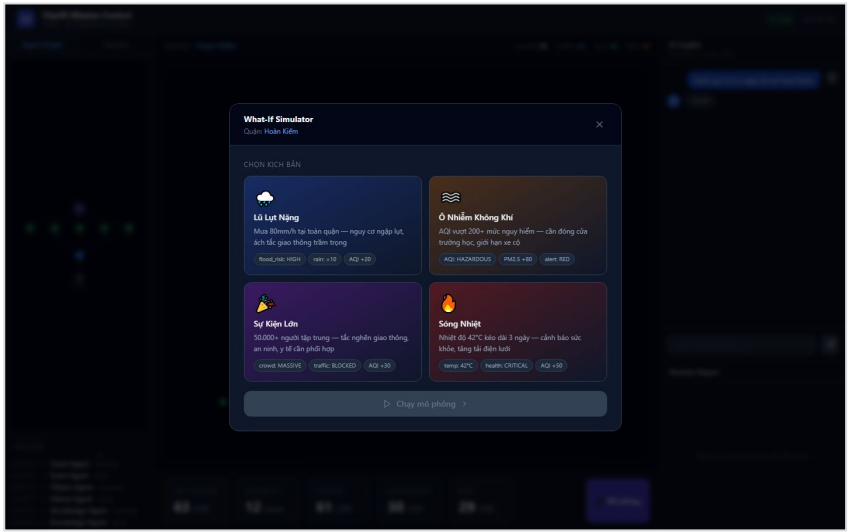
Google: Gemini 2.0 Flash — v1 pipeline + automatic fallback when Nemotron/OpenRouter is unavailable

Knowledge: Neo4j Aura (entity/relation graph) · Qdrant Cloud (vector search)

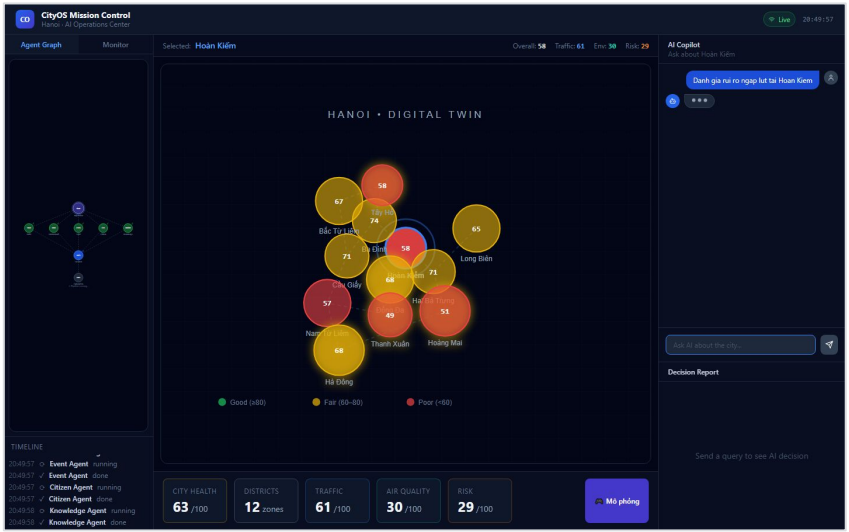
Platform: FastAPI + PostgreSQL (Neon.tech) · React + TypeScript · Render (backend) · Vercel (frontend)

Why this stack: every model call routes through one AI Gateway (src/ai/gateway.py) with a per-role fallback list — changing providers (NVIDIA NIM, Vertex AI, Ollama) means editing one file, not every agent. 100% free-tier components keep the stack deployable at zero GPU cost, and the gateway abstraction makes production scale-up a config change, not a rewrite.

Snapshots of the prototype



What-If Simulator — scenario testing



Mission Control — live agent run with AI Copilot

Google Cloud H2S

Gen AI Academy

APAC Edition

Build in APAC. Build for the world.

Thank you